

Session 3 · Deck 3.B3

PyTorch from the Ground Up, and Coding with AI

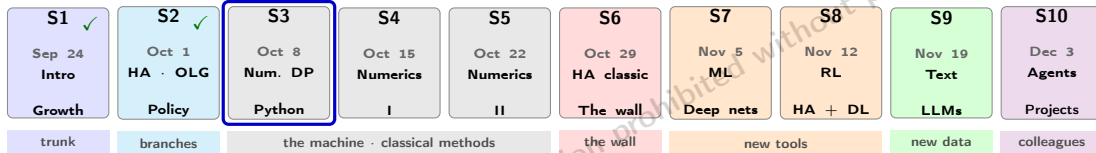
Zhigang Feng

Thursday, October 8, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 3.B3 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 3.B1 your toolkit · 3.B2 from loops to arrays · 3.B3 PyTorch, and coding with AI · 3.B4 the data workflow · 3.B5 hands-on: build it yourself.

Hands-on, 3.B5:

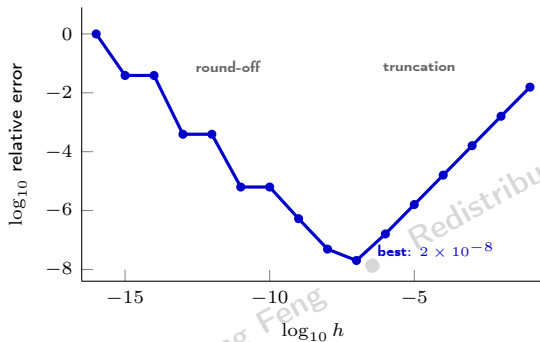
1 Version 0: grid VFI
against the closed form

2 Data → parameters:
 $g, n, \alpha, \delta, \beta$

3 The calibrated economy,
solved and checked

4 Test the model: untargeted
ratio, Euler residual

A Derivative You Cannot Get Right by Hand



The forward difference of k^α at $k = 2$. Shrink h and the approximation improves — until round-off takes over, and at $h = 10^{-16}$, $2 + h$ rounds to exactly 2, the numerator is zero, and the answer is **100% wrong**.

The best you can do is about 2×10^{-8} , at $h = 10^{-7}$ — near $\sqrt{\epsilon_{\text{mach}}}$ — and you must know that in advance.

`x.grad` → relative error **exactly 0**.

Not a better approximation — *the same arithmetic the chain rule would give you*, done by the library.

What PyTorch Actually Is

Four things, and only the first two matter today:

1. **Tensors** — NumPy arrays that also run on a GPU.
2. **Autograd** — a record of every operation, so derivatives come back exactly.
3. `nn.Module` — a class for holding parameters.
4. `optim` — gradient-descent variants.

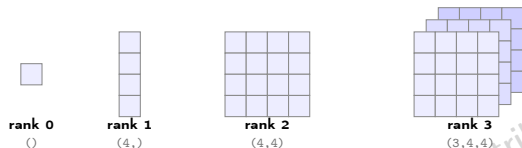
It is a scientific-computing library that happens to be popular for deep learning, not the other way round.

Why an economist needs it

Every method from here on rests on a derivative: Newton needs a Jacobian, perturbation needs high-order derivatives at the steady state, and a neural solver is trained by differentiating a residual.

And carry 3.A1's warning in: PyTorch defaults to `float32`. For anything with an accounting identity in it, set `dtype=torch.float64` explicitly.

Tensors



A scalar, a vector, a matrix and a 3-tensor are one object at different ranks.

shape is the tuple of axis lengths.

```
import torch
```

```
k = torch.linspace(0.05, 0.6, 200,  
                    dtype=torch.float64)  
k.shape, k.dtype      # (200,) float64
```

```
A = torch.zeros(200, 200)  
A @ A                    # matrix product  
k[:, None] - k[None, :] # broadcasting
```

```
k.numpy()                # -> NumPy  
torch.from_numpy(arr)    # <- NumPy
```

Broadcasting rules are **identical** to NumPy's. Everything from 3.B2 transfers unchanged.

reshape copies when it must; view reuses memory and refuses when it cannot. **Use** reshape unless you have a reason — and either can return a **view**, which is 3.B1's copy trap one library on.

Autograd: The Tape

```
x = torch.tensor(2.0, requires_grad=True,
                 dtype=torch.float64)
y = x ** 0.36

y.backward()      # walk the tape back
x.grad            # 0.23101666156132275

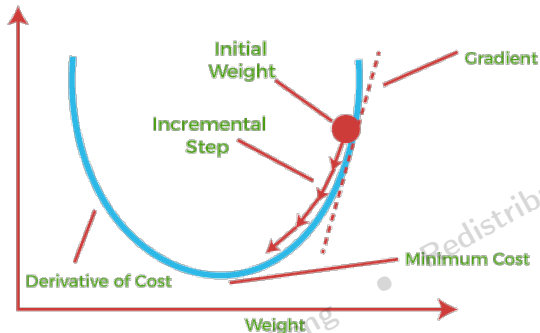
# exact: 0.36 * 2**(-0.64)
#         = 0.23101666156132275
```

Marking a tensor `requires_grad` makes PyTorch **record** every operation it takes part in: nodes are tensors, edges are operations.

`backward()` walks that graph in reverse, applying the chain rule one operation at a time. Each step is exact to machine precision, so the result is the derivative — not an approximation of it.

Gradients **accumulate**. Forget `zero_grad()` and you are descending on the sum of every gradient so far.

Descent, in One Picture



The loss surface, and the step $\theta \leftarrow \theta - \eta \nabla_{\theta} L$.

Once derivatives are free, one algorithm covers an enormous amount of ground: write down a scalar you want small, and walk downhill.

Fit a curve	squared prediction error
Solve a model	squared Euler residual
Train a net	either of the above

The middle row is the whole idea of Session 7: the residual of 3.A1 becomes a **loss**, and the policy function becomes the thing being trained.

The Training Loop, Which Is Always the Same Five Lines

```
model = nn.Linear(4, 1)
opt    = torch.optim.Adam(model.parameters(),
                           lr=1e-3)

lossf = nn.MSELoss()

for epoch in range(n_epochs):
    opt.zero_grad()           # 1. clear
    pred = model(x)           # 2. forward
    loss = lossf(pred, y)      # 3. score
    loss.backward()            # 4. backward
    opt.step()                 # 5. update
```

Memorise the five steps. Every PyTorch program you will read has them, in this order, and almost every bug is one of them missing.

The most common bug in the world is omitting `zero_grad()`. It does not crash. It trains, slowly and wrongly, and the loss curve looks almost plausible.

Compare it with 3.A2's loop: guess, evaluate, measure, update, repeat. Same skeleton — only the update rule changed, from a max to a gradient step.

A Network Is a Class — You Already Know This Pattern

```
class PolicyNet(nn.Module):
    def __init__(self, width=32):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2, width), nn.ReLU(),
            nn.Linear(width, width), nn.ReLU(),
            nn.Linear(width, 1))

    def forward(self, s):
        return self.net(s)

net = PolicyNet()
sum(p.numel() for p in net.parameters())
# 1185
```

Exactly the structure of GrowthModel:

`__init__` sets up the state, a method supplies the behaviour. Inheriting from `nn.Module` is what makes the parameters findable by the optimiser.

Read the input and output dimensions: a function of the state (k, z) returning one number. That is a **policy function** — represented by 1,185 numbers instead of a grid.

Which is why this matters for us: the representation no longer grows like N^d . That is the door out of the curse, and Sessions 7 and 8 walk through it.

The Five Pillars You Have to Supply

Pillar	What you must be able to do	
Economic theory	objective, constraints, optimality conditions, equilibrium concept	S1–S2
Algorithm design	turn that into an operator: Bellman or Euler, value or policy iteration	S3
Numerical technique	optimization, interpolation, discretizing an AR(1), simulation	S4
Advanced methods	perturbation (fast, local) against projection (global, costly), and when each is right	S5
Code literacy	read a VFI loop against your design; spot a wrong index or a missing constraint	S3, labs

You	AI
the question,	correct syntax
the model	
the algorithm	the translation
the method	the routines
the accuracy criteria	the diagnostics

The bottleneck moved: it was *can you write the loop*, and it is now **can you say what the loop must satisfy, and tell whether it does**. Both are the left-hand column — and both are how you diagnose a wrong answer.

A Workflow That Works

1. **Design first, separately.** Settle the model, the algorithm and the diagnostics before any code is requested.
2. **Start at version 0** — the case with a closed form. Ours is $\delta = 1$, log utility.
3. **Implement** one version, with the diagnostics built in from the start.
4. **Validate** against the benchmark before adding anything.
5. **Extend** by one feature, and re-validate.

Why this order

Every step keeps a working, checkable program in your hands. When version 2 breaks, the only thing that changed is the one feature you added.

The failure mode is asking for the full heterogeneous-agent model on the first prompt. You get 300 plausible lines, no benchmark, and no way to find out which part is wrong.

Start at Version 0, Where You Know the Answer

Goal: a stochastic RBC model with variable labour.
Do not start there.

V	Adds	Benchmark
0	deterministic, fixed labour	closed-form steady state
1	stochastic productivity	$\sigma \rightarrow 0$ recovers V0
2	variable labour supply	limiting case is V1

$k^* = \left(\frac{\alpha\beta}{1 - \beta(1 - \delta)} \right)^{\frac{1}{1-\alpha}}$ — and at $\delta = 1$ that is
 $(\alpha\beta)^{1/(1-\alpha)} = 0.187$, **this morning's model.**

Step 1: design, before any code

"Design a VFI algorithm for the deterministic RBC model with: (1) production $Y = K^\alpha$ and depreciation δ ; (2) Bellman equation $V(k) = \max_{k'} \ln(K^\alpha + (1 - \delta)K - k') + \beta V(k')$; (3) grid search over k' ."

Then, *before* implementing: "Can this design extend to (a) stochastic productivity with state (k, A) , (b) labour as a second choice? Show where each would enter."

Then Implement — and Validate, Which Is Not the Same Step

Step 2: implement, with diagnostics

“Implement this algorithm in Python. Include: (1) a convergence plot of $\|V^{n+1} - V^n\|_\infty$ against iteration; (2) the numerical k^* against the analytical formula; (3) the policy $k'(k)$ with the 45-degree line; (4) Euler-equation residuals.”

The diagnostics are requested **in the same breath as the code**. Asking for them afterwards is how you end up with a program you cannot check.

Step 3: validate — your job

- Does $k'(k)$ cross the 45-degree line at the analytical k^* ?
- Are the Euler residuals where they should be for the method? (Grid search: no. Time iteration: yes.)
- Do the comparative statics have the right sign — higher β , higher k^* ?

Steps 4–5: extend, then re-validate. Add the AR(1) shock; check that $\sigma = 0$ reproduces Version 0 *exactly*. Only then add labour.

Validation Is the Part You Cannot Delegate

Technical checks

- Does it converge, and in the expected number of iterations?
- Does it match the closed form?
- Are the Euler residuals off the grid as small as the method promises?
- Does the answer move when the grid or tolerance moves?

Economic checks

- Is the policy monotone? Is c increasing in wealth?
- Is k^* near the analytical steady state?
- Do the comparative statics have the right *sign*?
- Is the magnitude plausible next to data?

AI writes the code. You validate the economics. An assistant can run every technical check in the left column; it has no way to notice that a 90% capital share is absurd, and it will never tell you that the question was not worth asking.

Takeaways

- Finite differences bottom out at about 2×10^{-8} and then get *worse* — the round-off floor of 3.A1, in the one place it does most damage. **Automatic differentiation has no such floor**: it applies the chain rule, exactly.
- Tensors are NumPy arrays with a tape attached. Broadcasting, views and the copy trap all carry over unchanged. **Set float64 yourself** — PyTorch will not.
- The five-line training loop — zero, forward, score, backward, step — is every PyTorch program you will ever read, and `zero_grad()` is the line people forget.
- A network is a class holding parameters, and a policy net is a function of the state with 1,185 numbers instead of a grid. That is the escape route from the curse, and the reason Block B ends here.
- **Specifying a problem precisely and understanding it are the same act**. Read the RBC design prompt again — it *is* 3.A1's six decisions with a benchmark attached, which is why a vague prompt is a symptom rather than a habit.

Bridge: The Model Gave You Numbers. So Does the World.

Everything today produced numbers out of a *model*: a policy function, an iteration count, a residual. Every one was graded against a closed form, because we had one.

Outside this benchmark you do not. What you have instead is **data** — and a model earns its keep by reproducing moments the data actually shows. That comparison has its own workflow, and it is the next deck.

Next (3.B4)

Acquire, clean, persist, analyse, export — and the HP filter, which is how a simulated series and a real one are made comparable.

It is also the data half of **Homework 1** (Part A, A3) that Block A did not give you.